

자바스크립트의 이해



JavaScript, ECMAScript

브라우저에서 화면이 그려지는 과정

HTML은 구조를 만들고, CSS는 모양을 정하고, JavaScript는 동작을 만든다.



JavaScript가 혼자서 화면을 바꾸는 것은 아니다.

JavaScript Engine이 코드를 실행하고 → Web API를 이용해 DOM을 변경하고 → Rendering Engine이 DOM과 CSSOM을 이용해 화면을 다시 그린다.



HTML = 구조

웹 페이지의 내용과
요소들의 구조를 정의한다.



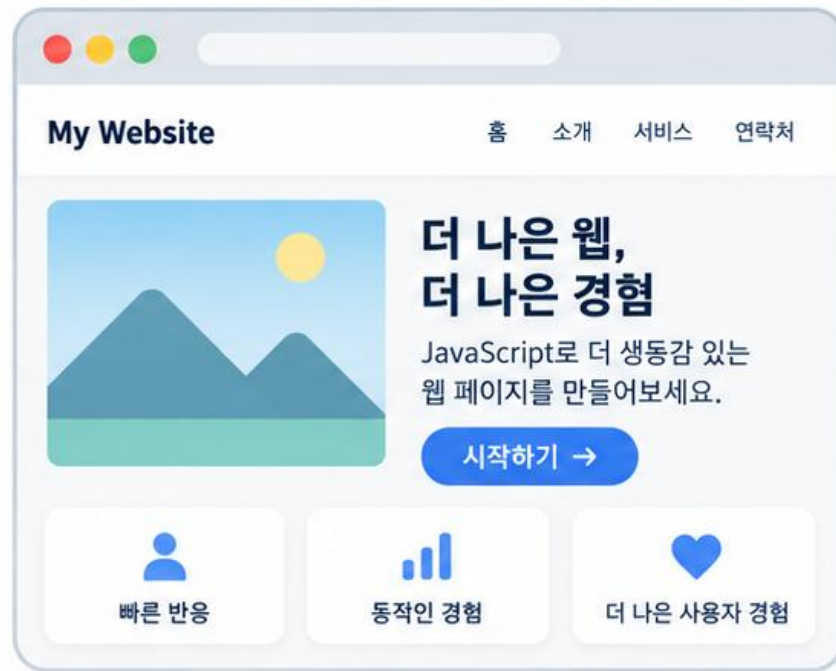
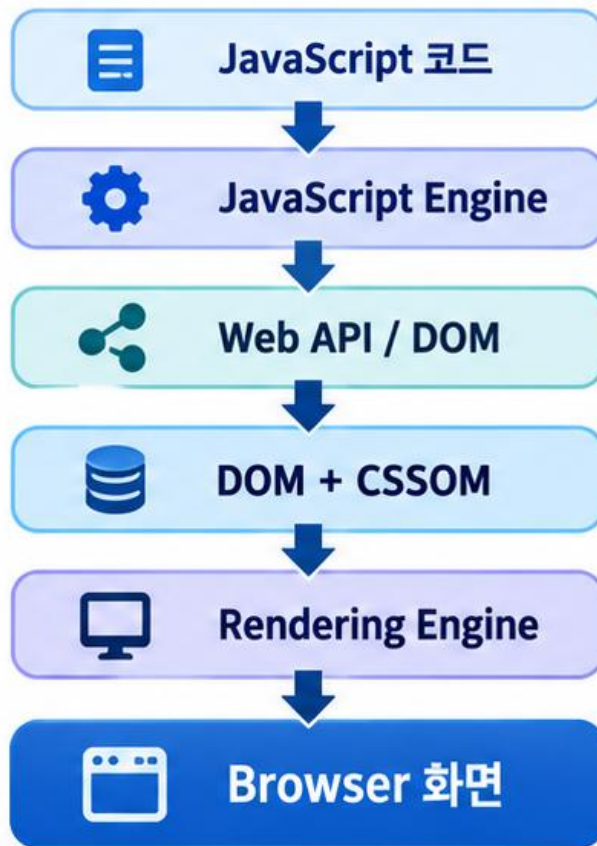
CSS = 모양

요소의 크기, 색상, 배치 등
화면의 스타일을 설정한다.



JavaScript = 동작

사용자와의 상호작용을 처리하고,
화면의 내용을 동적으로 변경한다.



브라우저는 DOM과 CSSOM을 바탕으로
화면을 다시 렌더링한다.



이번 강의의 핵심: JavaScript 문법만이 아니라, 브라우저 내부 동작 구조를 이해하는 것

자바스크립트란?

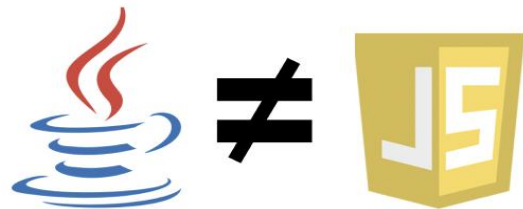
- 자바스크립트(JavaScript)는 '웹페이지에 생동감을 불어넣기 위해' 만들어진 프로그래밍 언어
 - 자바스크립트로 작성한 프로그램을 스크립트(Script)라고 함
 - 스크립트는 웹 페이지의 HTML 안에 작성할 수 있으며, 웹페이지를 불러올 때 스크립트가 자동으로 실행
 - JavaScript는 개발자가 별도의 사전 컴파일 명령을 수행하지 않아도 브라우저에서 실행 가능
 - JavaScript 엔진 내부에서는 소스코드의 분석, 컴파일 및 실행 과정이 수행

왜 자바스크립트인가?

95년 넷스케이프사에서 개발 당시 코드네임 모카(Mocha)로 시작 되었으며 그 이후 '**Live Script**'라고 불려 졌음
당시 최고 인기 언어였던 썬 마이크로시스템즈의 자바(JAVA) 인기에 편승하기 위하여
마케팅 전략에 따라 **자바스크립트**로 최종 이름이 바뀜

자바와 자바스크립트 관계

이름이 비슷하지만 기술적이나 언어 구조적으로 직접적인 연관 거의 없음
자바는 정적 객체지향 언어, 자바스크립트는 동적 스크립트 언어임
꾸준히 발전하여 **ECMAScript**라는 고유한 명세를 갖춘 독립적인 언어가 됨



JavaScript 역할 예

- HTML만 있는 웹페이지
 - 버튼은 존재하지만 아무 일도 발생하지 않음

```
<button>클릭</button>
```

- JavaScript를 연결하면
 - 사용자의 행동에 반응할 수 있음

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  alert("버튼을 클릭했습니다.");
});
```

HTML은 버튼이라는 객체를 생성

CSS 는 버튼의 모양을 정함

JavaScript는 버튼이 눌렸을 때 무엇을 할지 결정

- JavaScript가 직접 버튼을 찾아가는 것이 아니고 브라우저가 만들어 놓은 DOM API를 이용해서 HTML 요소에 접근

- ECMAScript(ES, 에크마스크립트) - European Computer Manufacturers Association
 - 현재 조직 명칭은 Ecma International
 - ECMAScript는 JavaScript가 따라야 하는 표준 명세
 - ECMA International의 기술위원회 TC39(Technical Committee 39)가 언어의 문법, 데이터 타입, 연산자, 함수, 객체, 클래스, 모듈, 실행 규칙 등을 정의 → 그 결과가 ECMA-262라는 기술 규격 표준으로 발행
 - ES6(ES2015)는 ECMAScript 6 이라는 의미로 ECMA-262 표준의 제 6판 이라는 뜻, ES6+는 ES6 이후를 의미



ECMAScript는 JavaScript라는 언어의 **설계도(규칙)**이고, **JavaScript 엔진**은 그 설계도를 구현한 **실행 프로그램**이다.

ECMAScript와 JavaScript 엔진의 관계

ECMAScript는 문법의 설계 규격이고, JavaScript 엔진은 그 규격을 구현하여 코드를 실행한다.



브라우저 회사가 let, const, 화살표 함수, class 문법을 마음대로 만드는 것이 아니다.
ECMAScript 명세를 기준으로 구현한다.



ECMAScript = 표준

JavaScript의 문법과 동작 원칙을 정의하는 공식 명세(규격)이다.



ECMAScript = 자동차 설계 규격

문법과 동작 원칙을 정의하는 표준 명세



규격에 따라 구현



V8



SpiderMonkey



JavaScriptCore

= 실제 구현된 JavaScript 엔진

브라우저와 런타임에서 ECMAScript를 해석·실행



코드 실행



JavaScript Code

개발자가 작성한 코드



ECMAScript에서 정하는 대표 문법 예시

```
let name = "Kim";
const age = 20;
const add = (a, b) => a + b;
class Student {
}
```

이러한 문법은 특정 브라우저 회사가 임의로 만든 것이 아니라, ECMAScript 명세를 기준으로 각 엔진이 구현한 것이다.



자동차에 비유하면?



설계 도면

: ECMAScript (규격)



자동차 제조사

: JavaScript 엔진 (구현체)



실제로 달리는 자동차

: JavaScript 코드 (실행되는 결과)



이번 강의의 핵심: JavaScript는 독립적으로 존재하는 것이 아니라, ECMAScript 표준과 JavaScript 엔진 위에서 동작한다.



- ECMAScript는 JavaScript의 핵심 언어 영역 및 표준
 - ECMAScript는 각 **JavaScript 엔진이 공통된 문법과 실행 규칙을 구현하도록** 하는 기준
 - ECMAScript 표준은 다음 내용을 정의
 - 변수 선언(let, const)과 유효범위
 - 데이터 타입(number, string, boolean, bigint)과 타입 변환
 - 연산자와 제어문(if, for, while)
 - 함수와 화살표 함수, 배열과 배열 메서드
 - 객체와 클래스, 상속
 - Map, Set
 - 비동기 처리를 위한 Promise, async, await
 - 모듈의 import, export
 - 코드 실행 순서와 오류 처리
 - ECMAScript 언어 타입은 Undefined, Null, Boolean, String, Symbol, Number, BigInt, Object로 정의

- ECMAScript는 고정된 표준이 아니라 지속적으로 개선

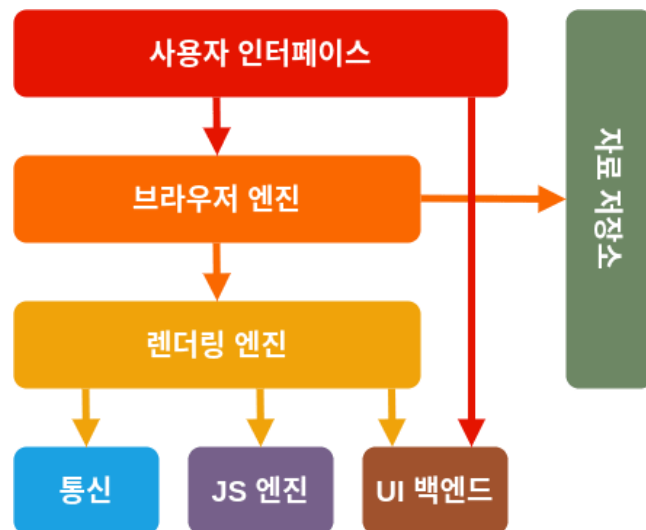
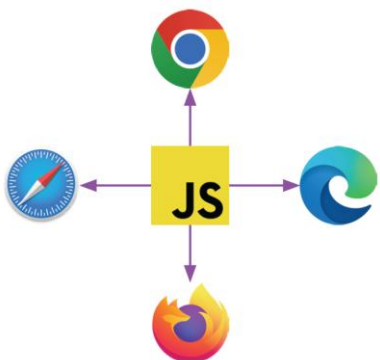
명칭	발표 연도	의미
ECMAScript 1	1997	최초 표준
ECMAScript 5, ES5	2009	엄격 모드, 배열 메서드 등
ECMAScript 6, ES6	2015	현대 JavaScript의 기반
ECMAScript 2016 이후	매년	연도 기반으로 표준 발표
ECMAScript 2026	2026년 6월	ECMA-262 제 17판, 현재 최신 확정 표준

- ECMAScript 와 JavaScript 실행 구조
 - ECMAScript 표준 **구현** → JavaScript 엔진 **실행** → JavaScript 코드 + 브라우저 Web API(DOM, 이벤트, fetch)

JavaScript 엔진

• JavaScript 엔진

- JavaScript 소스 코드를 컴퓨터가 실행할 수 있는 형태로 변환하고 실행하는 프로그램
 - JS 소스코드 -> JS 엔진 -> Parsing -> Compilation / Optimization -> Execution 과정으로 실행
 - 컴파일과 최적화(Compilation / Optimization) : JS 엔진이 코드 실행할 때 속도를 높이기 위한 핵심 기술
 - 크롬의 V8엔진은 실행 속도를 극대화하기 위해 JIT(Just-In-Time) 컴파일러와 최적화 컴파일러를 사용
- 브라우저는 HTML·CSS를 화면으로 렌더링하고, DOM(Document Object Model), 이벤트, fetch(), 타이머, 저장소 같은 기능을 JavaScript 코드에서 사용할 수 있도록 제공
- V8**이 ECMAScript와 WebAssembly를 구현하고, JavaScript 코드의 컴파일 · 실행 · 메모리 관리 · 가비지 컬렉션을 담당



브라우저	렌더링 · 브라우저 엔진	JavaScript 엔진
Google Chrome	Blink	V8
Microsoft Edge	Blink	V8
Opera	Blink	V8
Brave	Blink	V8
Mozilla Firefox	Gecko	SpiderMonkey
Apple Safari	WebKit	JavaScriptCore

- **엔진의 종류**

- **V8 (Chrome, Node.js)**

- Google에서 개발한 오픈소스 엔진, C++로 개발되었으며 빠른 실행 속도와 높은 성능을 갖추, 주로 Chrome 브라우저와 Node.js 런타임 환경에서 사용

- **SpiderMonkey (Firefox)**

- Mozilla에서 개발한 엔진, 최초의 자바스크립트 엔진으로 안정성과 호환성에 중점을 둔 엔진, 주로 Firefox 브라우저에서 사용

- **JavaScriptCore (Safari)**

- Apple에서 개발한 엔진으로 Nitro 또는 SquirrelFish Extreme라는 이름으로도 알려짐, MacOS 및 iOS 환경의 Safari 브라우저에서 사용

- **Chakra (Microsoft Edge Legacy)**

- Microsoft에서 개발한 엔진, 이전 버전의 Microsoft Edge 브라우저에서 사용, 현재는 Chromium 기반 Edge 브라우저에서 V8 엔진 사용

- **Hermes (React Native)**

- Facebook에서 개발한 엔진, 모바일 환경에서의 빠른 시작 시간과 적은 메모리 사용량에 최적화하는 장점, React Native 애플리케이션에서 사용

자바스크립트 엔진과 렌더링 엔진의 차이

- JavaScript 엔진은 프로그램의 동작을 실행
- 렌더링 엔진은 HTML과 CSS를 사용자가 보는 화면으로 변환
- **JavaScript 코드 실행 → DOM API 호출 → DOM·스타일 변경 → 렌더링 엔진 → 화면 업데이트**
 - Chrome Browser 의 blink : 화면 렌더링
 - Chrome Browser 의 V8 : JavaScript 실행

구분	JavaScript 엔진 (프로그램을 실행)	렌더링 엔진 (화면을 그림)
주요 입력	JavaScript 코드	HTML, CSS, DOM, CSSOM
주요 역할	코드 분석·실행	화면 구조와 스타일 계산
메모리 관리	Call Stack, Heap, GC	DOM, 스타일, 렌더링 정보 관리
주요 결과	연산 결과, 함수 실행, DOM 변경 요청	화면의 위치·크기·색상·그래픽 출력
대표 사례	V8 , SpiderMonkey, JavaScriptCore	Blink , Gecko, WebKit
직접 화면 출력	하지 않음	담당함
DOM 제공	하지 않음	브라우저 런타임과 웹 엔진 영역에서 제공

자바스크립트 엔진과 렌더링 엔진의 차이

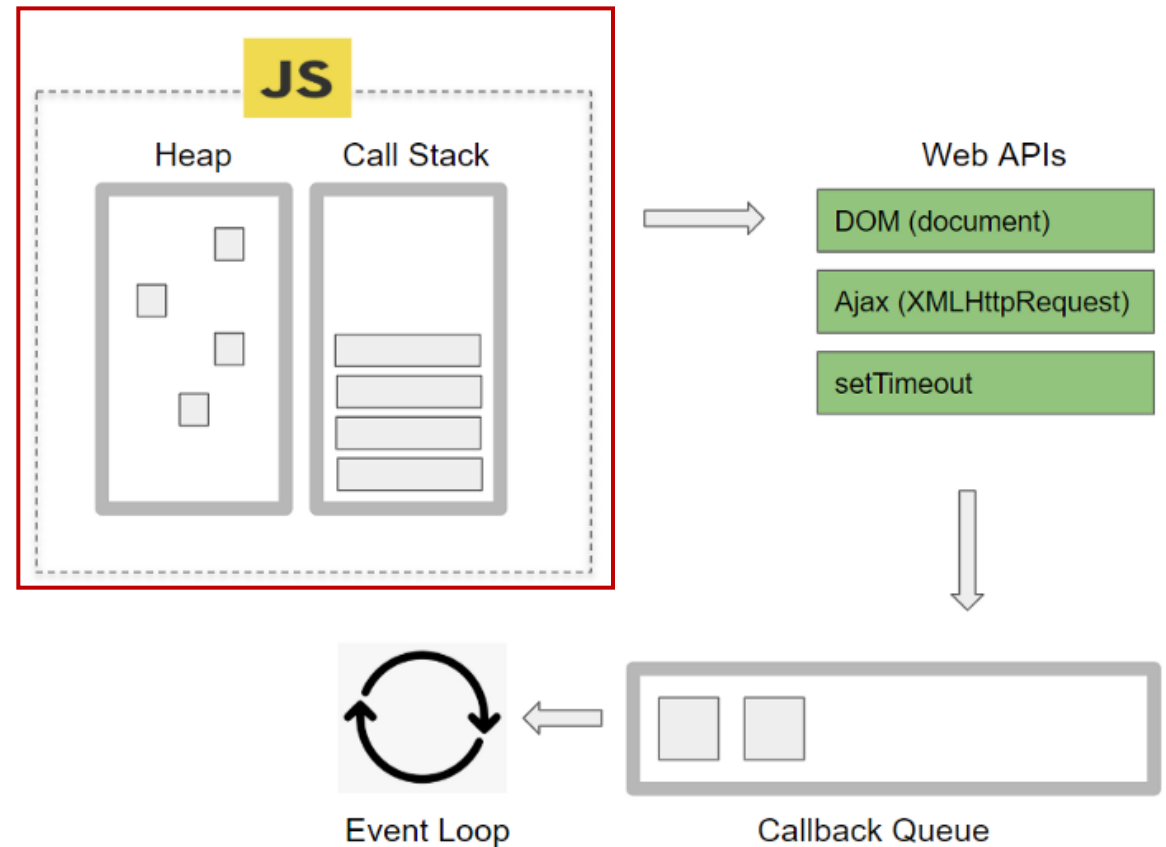
- JavaScript 코드 실행 이해
 - JS 소스코드 -> JS 엔진 -> Parsing -> Compilation / Optimization -> Execution 과정으로 실행
- **`document.querySelector("#title").textContent = "Hello";`**
 - JavaScript 엔진이 직접 화면에 Hello 라는 글자를 그리는 것이 아님
 - 실제 흐름
 - JavaScript 실행 -> DOM API 호출 -> DOM 변경 -> Rendering Engine -> 화면 변경

• 주요 역할

- 소스 코드 읽기, 문법 분석, 실행 가능한 중간 형태로 변환
- 코드 실행, 반복적으로 실행되는 코드 최적화
- 객체와 변수의 메모리 관리, 사용하지 않는 메모리 회수

• 주요 구성 요소

- 콜 스택(Call Stack)
 - 함수 호출 순서를 관리하고 현재 실행 중인 함수의 정보를 저장하는 자료구조(LIFO)
- 메모리 힙(Memory Heap)
 - 역할 : 객체, 배열, 함수 등 동적으로 생성되는 데이터를 저장하는 메모리 공간
 - 구조 : 힙은 상대적으로 크고 구조화되지 않은 메모리 영역으로 필요에 따라 메모리를 할당하고 해제
 - 가비지 컬렉션 : 가비지 컬렉션은 더 이상 접근할 수 없는 객체를 찾아 해당 객체가 사용하던 메모리를 자동으로 회수



브라우저 내부 구성도

Call Stack과 Heap

간단한 코드로 이해하는 JavaScript 실행 흐름



Call Stack은 현재 JavaScript가 어디까지 실행되고 있는지 관리하는 공간이다.

</> 예제 코드

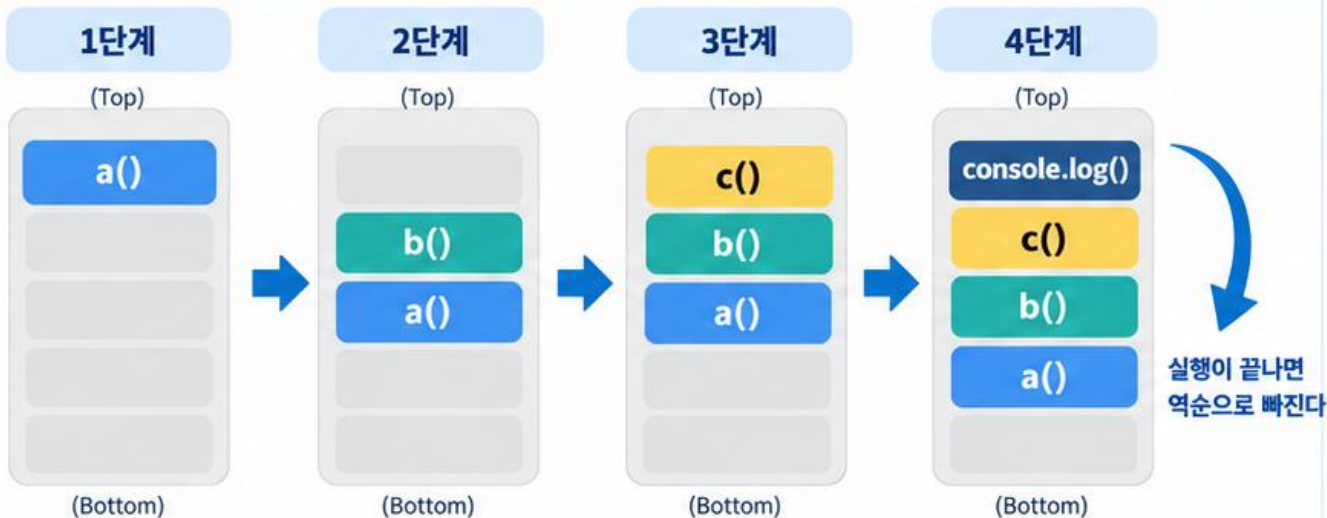
```
JavaScript
function c() {
  console.log("C");
}

function b() {
  c();
}

function a() {
  b();
}

a();
```

Call Stack 변화



🎯 핵심 개념



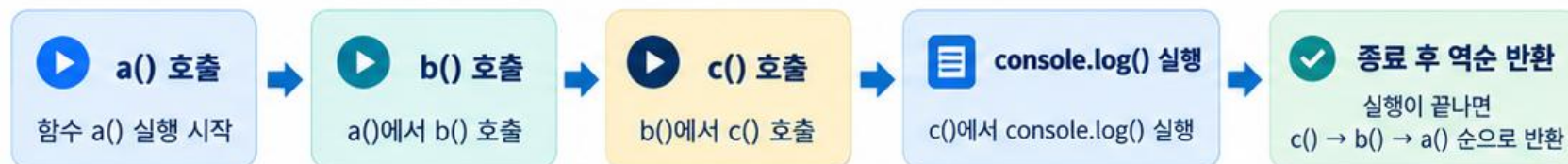
LIFO

Last In, First Out

후입선출

가장 나중에 들어온 함수가 가장 먼저 실행을 마치고 빠져나간다.

⚙️ 실행 흐름 요약



⚖️ Call Stack과 Heap 비교



Call Stack

함수 호출 순서와 현재 실행 위치를 관리



Heap

객체, 배열 등 참조 데이터가 저장되는 메모리 공간

※ 이번 강의에서는 Call Stack에 초점을 맞춥니다.



이번 강의의 포인트: Call Stack은 함수 호출과 실행 순서를 추적하며, 함수는 스택에 쌓였다가 LIFO 방식으로 제거된다.



> Heap이란? <

객체 같은 동적 데이터가 저장되는 메모리 공간



이번 과정에서는 Heap을 '객체 등의 데이터가 저장되는 공간' 정도로 이해하면 충분합니다.

</> 예제 코드

JavaScript

```
const student = {  
  name: "Kim",  
  age: 20  
};
```

student 객체가 생성되면,
이 객체 데이터는 **Heap 영역**에 저장된다.



Heap에 저장되는 것



객체, 배열 등 참조 데이터가 저장되는 영역



Call Stack과 Heap 구분



Call Stack

→ 함수 실행 관리



Heap

→ 객체 등의 데이터 저장



Garbage Collection



더 이상 사용할 수 없는 객체를
JavaScript Engine이 찾아서
메모리를 회수하는 기능

즉, 필요 없어지면 엔진이 자동으로 정리해 준다.



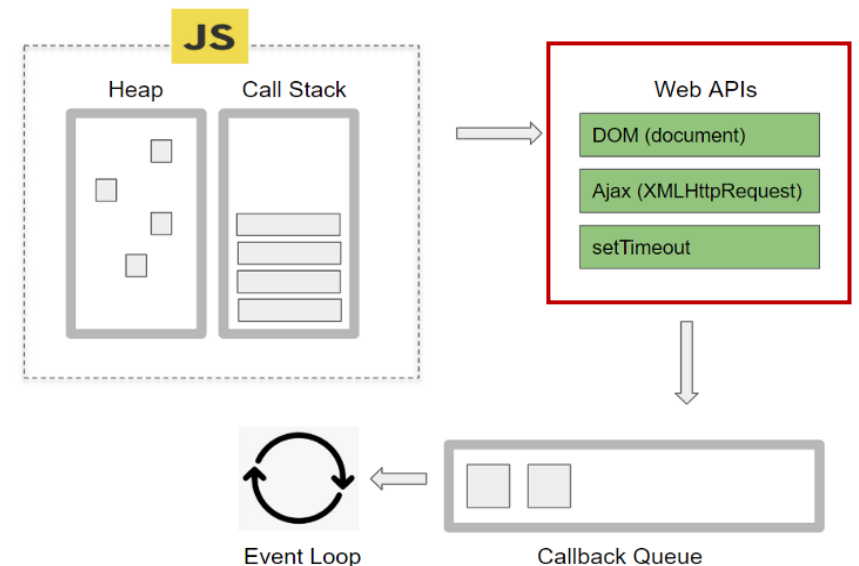
핵심 정리: Heap은 객체 같은 동적 데이터가 저장되는 공간이다.

Web APIs

• Web APIs

- 브라우저가 JavaScript 코드에서 문서, 이벤트, 네트워크, 저장소, 타이머, 그래픽 등의 기능을 사용할 수 있도록 제공하는 **API 집합**

영역	대표 API	주요 역할
문서·이벤트	DOM API, Event API	요소 조회·변경, 사용자 이벤트 처리
네트워크	Fetch API	서버에 데이터 요청
시간	Timer API	함수 실행 지연·반복
저장소	Web Storage API	브라우저 데이터 저장
그래픽	Canvas API	그래픽과 애니메이션
위치	Geolocation API	사용자 위치 정보 요청



브라우저 내부 구성도

- 웹에서 사용하는 JavaScript 환경 = JavaScript + Browser Web APIs
 - JavaScript 엔진이 DOM(Document Object Model)과 BOM(Browser Object Model)을 제공하는 것은 아니다.

Event Loop

왜 setTimeout(0)이 바로 실행되지 않을까?

최소 지연시간과 Event Loop로 이해하는 JavaScript 비동기 실행



setTimeout(callback, 0)은 '0초 후 즉시 실행'이 아니라
최소 지연시간이 지난 후 실행 가능한 상태로 등록된다고 이해하는 것이 좋다.

</> 예제 코드

```
JavaScript
console.log("start");
setTimeout(() => {
  console.log("timeout");
}, 0);
console.log("end");
```

▶ 실행 결과

```
1 start
2 end
3 timeout
```

i timeout 콜백은 바로 실행되지 않고,
나중에 실행된다.

⚙ 실행 흐름



✓ 1) setTimeout 등록 → 2) 타이머 완료 → 3) Task Queue 이동 → 4) Stack이 비면 실행

🎯 핵심 개념



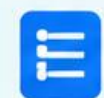
0 ms ≠ 즉시 실행

setTimeout(0)은 0초 후 바로 실행되는 것이 아니라, 최소 지연시간이 지난 후 실행된다.



Call Stack이 바쁘면, callback은 기다린다.

Call Stack에 실행 중인 코드가 있으면
setTimeout의 callback은 바로 실행되지 않고
대기한다.



Task Queue에 들어가도 바로 실행되는 것은 아니다.

타이머가 완료되면 콜백이 Task Queue(매크로태스크)에
들어가지만, 즉시 실행되지 않는다.



Event Loop가 Stack이 빈 순간 callback을 올린다.

Event Loop는 Call Stack이 비어 있는지 계속 확인하고,
비어 있는 순간 Task Queue의 콜백을 Call Stack으로
 옮겨 실행한다.

★ 한 줄 정리

1

setTimeout(0) = 즉시 실행 아님

2

최소 지연 후 대기열(Task Queue)로 이동

3

Call Stack이 비었을 때 실행



이번 강의의 포인트: setTimeout(0)은 바로 실행되는 것이 아니라, 대기열에 등록된 뒤 Call Stack이 비었을 때 실행된다.



동기, 비동기 처리

- 동기 (Synchronous)
 - 순차적 진행: 앞선 작업이 끝나야만 다음 작업을 시작
 - 직관적인 흐름: 작업의 순서가 명확해 설계와 디버깅이 단순
 - 대기 시간 발생: 결과를 받을 때까지 멈춰 있으므로 자원이 비효율적으로 쓰일 수 있음
- 비동기 (Asynchronous)
 - 독립적 진행: 작업의 완료 여부와 상관없이 즉시 다음 작업을 수행
 - 효율적인 시간 관리: 여러 일을 동시에 처리해 전체 소요 시간을 줄임
 - 복잡한 구조: 작업 완료 시점을 관리하기 위해 별도의 처리가 필요할 수 있음

JavaScript의 비동기 처리란?

JavaScript는 한 번에 하나씩 실행되지만, 오래 걸리는 작업은 브라우저에 맡긴다



JavaScript 코드 자체는 기본적으로 한 번에 하나씩 순차적으로 실행된다. 하지만 오래 걸리는 작업까지 모두 직접 처리하면 웹사이트가 멈춘 것처럼 느껴질 수 있다.



기본 실행 방식



JavaScript 실행은 기본적으로 순차적이다.



한 번에 하나씩 실행



비동기 처리의 기본 구조



브라우저의 다른 기능에 일을 맡기고, 완료되면 다시 JavaScript가 이어서 실행한다.



왜 필요한가?

네트워크 요청 예시



네트워크 요청 중...

네트워크 요청이 5초 걸린다고 해서 JavaScript가 5초 동안 멈춰 있으면 사용자는 매우 답답하게 느낀다.



그래서 오래 걸리는 작업은 Web API가 처리하고, 완료 후 Queue를 거쳐 다시 실행된다.



웹사이트가 멈추지 않도록 하기 위한 구조



이번 강의의 핵심: JavaScript는 기본적으로 순차 실행되며, 비동기 처리는 **Web API → Queue → Event Loop**를 통해 동작한다.

- **자바스크립트는 싱글 스레드 언어**

- 브라우저의 기본 JavaScript 실행 환경은 하나의 Call Stack을 사용하며, 한 시점에 하나의 JavaScript 작업을 실행

- **브라우저의 멀티 스레드와 Web APIs**

- 브라우저는 자바스크립트 엔진 외에도 여러 멀티 스레드 기능을 제공
- 파일 다운로드, 네트워크 요청, 타이머, 애니메이션 등의 작업은 브라우저의 Web API와 내부 시스템이 처리
- 작업이 완료되면 실행할 콜백이 적절한 Queue에 등록되고, Event Loop 가 실행 시점을 조정

- **이벤트 루프는 자바스크립트가 싱글 스레드 환경에서도 비동기 작업을 효과적으로 처리할 수 있도록 돕는 핵심 메커니즘**

- 이벤트 루프는 브라우저 내부의 Call Stack, Callback Queue, Web APIs 등을 모니터링하며 비동기 작업들을 관리

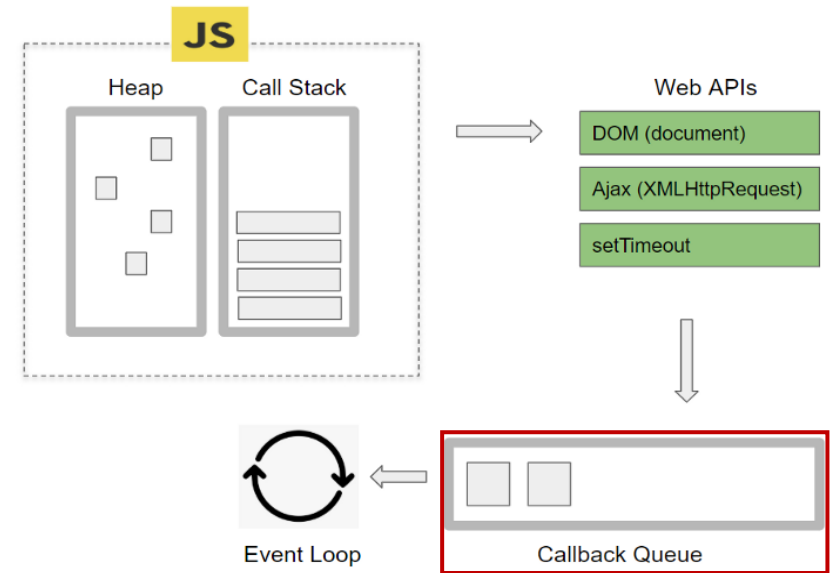
Task Queue와 Microtask Queue

• Callback Queue

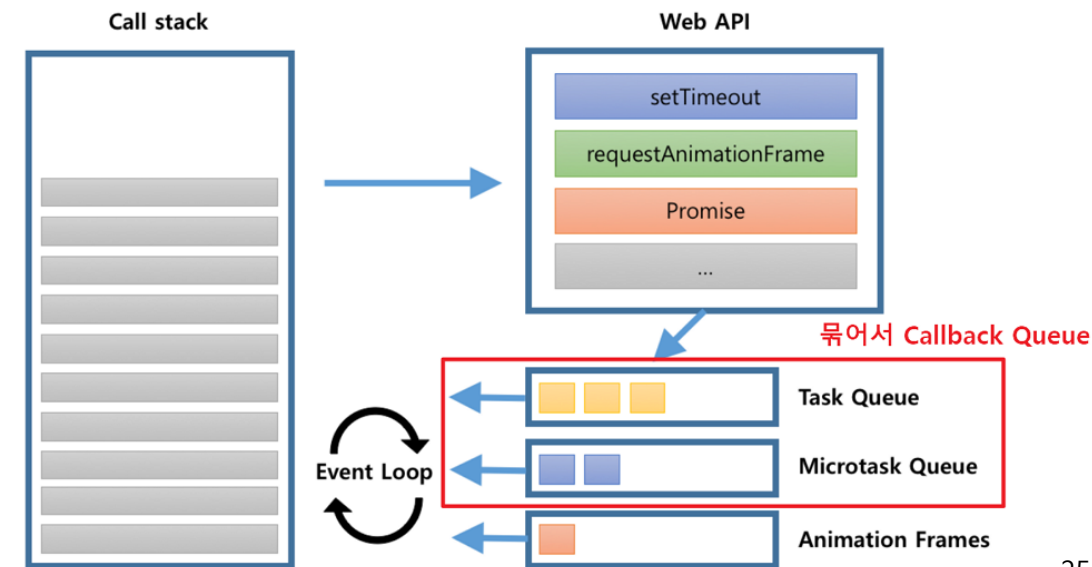
- 비동기 작업이 완료되면 실행되는 함수들이 대기하는 공간
- Task Queue(macrotask queue)
 - 역할: 비동기적으로 처리되는 함수들의 콜백 함수가 들어가는 큐
 - 예시: setTimeout, setInterval, 클릭·입력 등의 사용자 이벤트메시지 이벤트

• Microtask Queue

- 역할: 우선적으로 비동기 처리되는 함수들의 콜백 함수가 들어가는 큐
 - 예시: Promise.then(), Promise.catch(), Promise.finally(), queueMicrotask(), MutationObserver
-
- 이벤트 루프는 일반적으로 Microtask queue를 먼저 처리한 후, task queue를 처리
 - 따라서 Promise.then의 결과가 setTimeout보다 먼저 처리

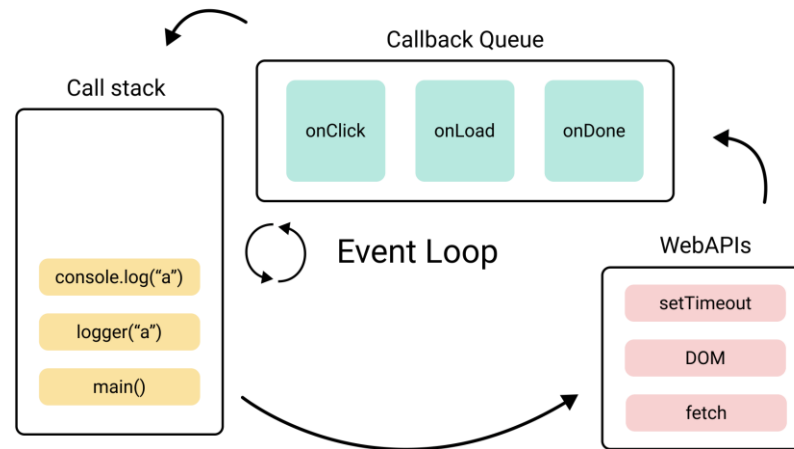


브라우저 내부 구성도



이벤트 루프의 동작 과정

1. Call Stack : 자바스크립트 함수 호출이 쌓이는 스택
2. Web APIs : 타이머, 네트워크 요청 등을 처리하는 브라우저의 API
3. Callback Queue : 비동기 작업이 완료되면 해당 콜백 함수가 이 큐에 추가
4. Event Loop : 호출 스택(Call Stack) 이 비어 있을 때 콜백 큐에서 콜백 함수를 꺼내 호출 스택에 추가하여 실행



비동기 프로그래밍은 네트워크 요청이나 타이머를 기다리는 동안

Call Stack이 불필요하게 차단되지 않도록 하여 웹페이지의 응답성을 유지하는 방식

이벤트 루프를 이용한 비동기 프로그래밍은 브라우저 환경에서 안정적인 실행과 높은 반응성을 제공

Promise와 setTimeout의 실행 순서

Microtask Queue와 Task Queue로 이해하는 JavaScript 비동기 처리



현재 실행 중인 JavaScript 코드가 끝나면, Event Loop는 **Task Queue** 보다 **Microtask Queue** 를 먼저 처리한다.

</> 예제 코드

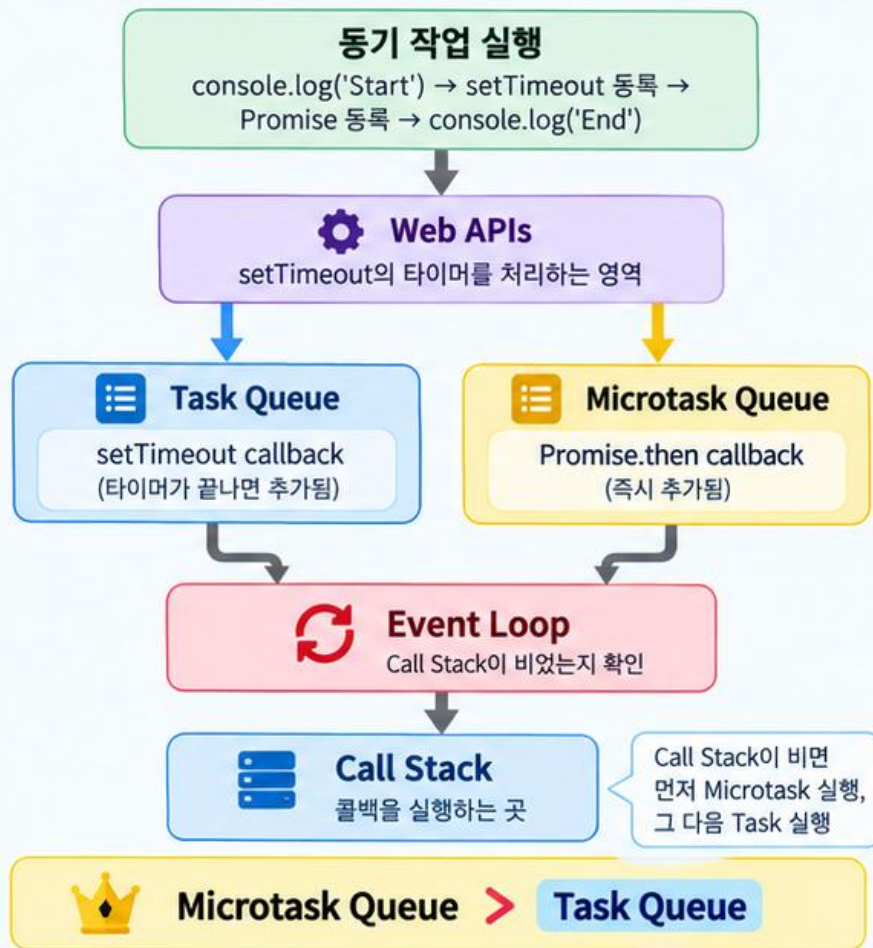
JavaScript

```
1 console.log('Start');
2
3 setTimeout(() => {
4   console.log('Timeout');
5 }, 0);
6
7 Promise.resolve().then(() => {
8   console.log('Promise');
9 });
10
11 console.log('End');
```

▶ 예상 출력 순서

- 1 Start
- 2 End
- 3 Promise
- 4 Timeout

⚙ 동작 구조



📄 설명

- 1 동기 작업 실행
콜 스택에 쌓인 일반 코드를 먼저 순차적으로 실행한다.
- 2 비동기 작업 처리
setTimeout은 Web APIs를 거쳐 Task Queue에, Promise.then은 Microtask Queue에 들어간다.
- 3 이벤트 루프 작동
Event Loop는 Call Stack이 비어 있는지 확인하고, 비어 있으면 대기 중인 콜백을 스택으로 옮긴다.
- 4 마이크로태스크 우선
Microtask Queue가 Task Queue보다 우선 처리되므로 Promise가 Timeout보다 먼저 실행된다.

📌 핵심 포인트 정리

- setTimeout 콜백 → Task Queue
- Promise 콜백 → Microtask Queue
- 우선순위: Microtask Queue 먼저



핵심 정리: Start와 End 같은 동기 코드는 먼저 실행되고, 그 다음 **Promise(Microtask)**, 마지막으로 **setTimeout(Task)**이 실행된다.

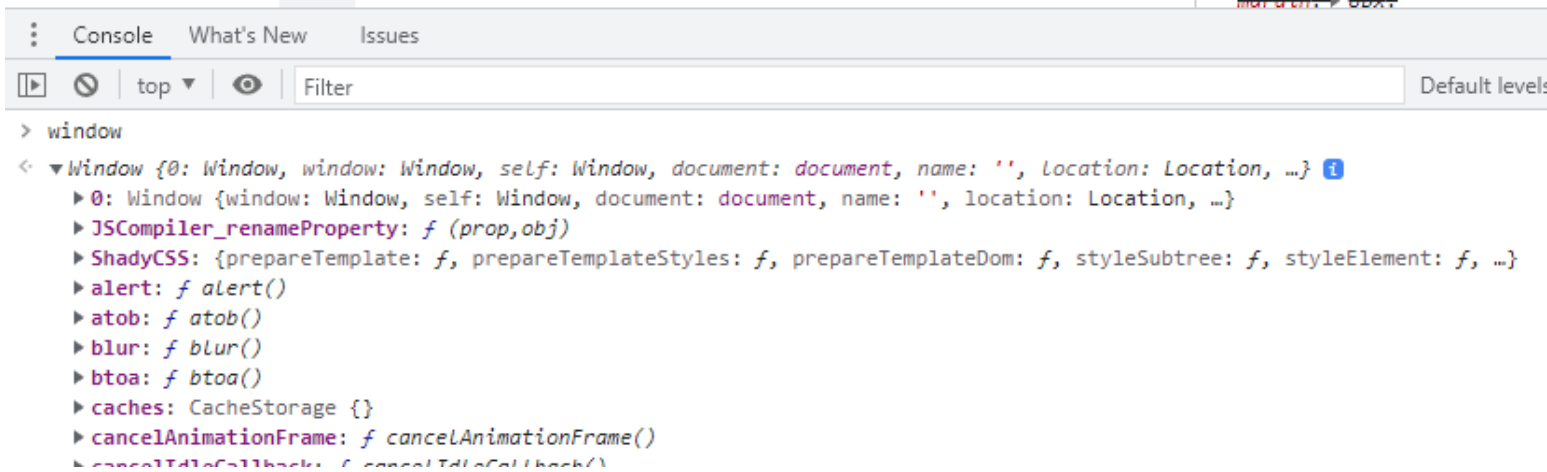


객체 구조

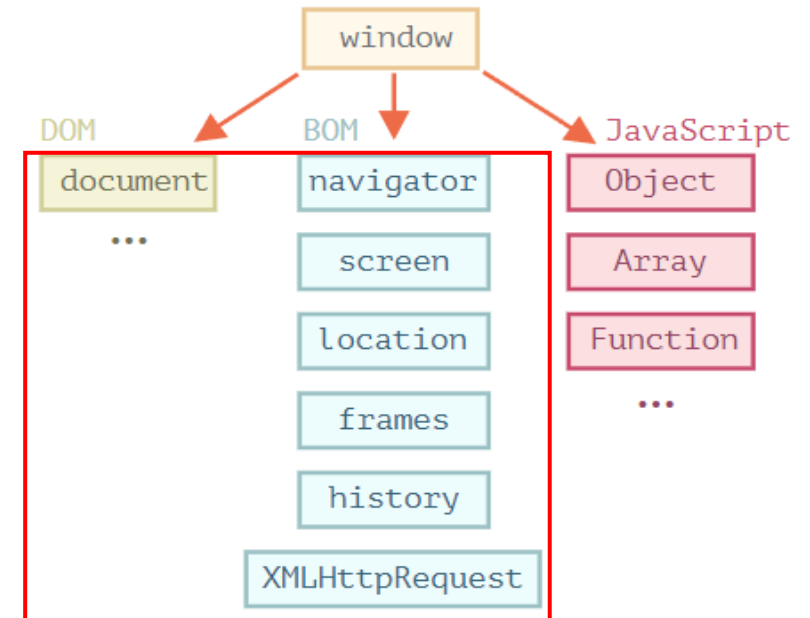
윈도우, DOM, BOM, CSSOM

윈도우(Window) 객체 구조

- 브라우저 환경에서 window는 현재 브라우저 창 또는 탭을 나타내며 **전역 객체의 역할**
 - window의 일부 프로퍼티와 메서드는 window를 생략하고 사용할 수 있음
 - 일반적으로 우리가 열고 있는 브라우저의 창을 의미, 이 창을 제어하는 다양한 메서드를 제공
- Window 객체의 프로퍼티나 메소드는 window를 생략하고 바로 사용할 수 있음
 - ex) window.alert("메시지") => alert("메시지")
 - ex) window.setTimeout() => setTimeout() 으로 사용



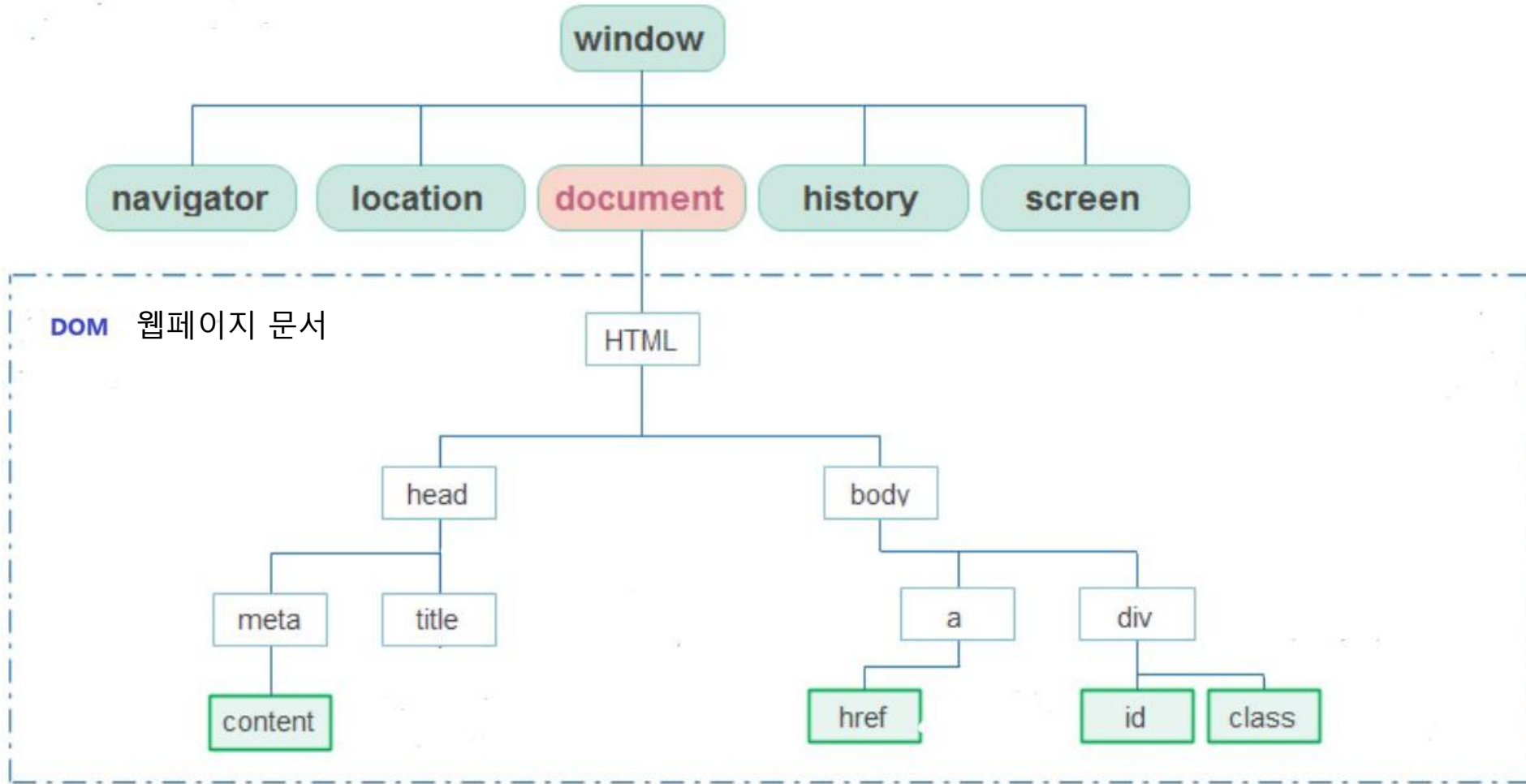
```
> window
< ▼ Window {0: Window, window: Window, self: Window, document: document, name: '', location: Location, ...} ⓘ
  ▶ 0: Window {window: Window, self: Window, document: document, name: '', location: Location, ...}
  ▶ JSCompiler_renameProperty: f (prop,obj)
  ▶ ShadyCSS: {prepareTemplate: f, prepareTemplateStyles: f, prepareTemplateDom: f, styleSubtree: f, styleElement: f, ...}
  ▶ alert: f alert()
  ▶ atob: f atob()
  ▶ blur: f blur()
  ▶ btoa: f btoa()
  ▶ caches: CacheStorage {}
  ▶ cancelAnimationFrame: f cancelAnimationFrame()
  ▶ cancelIdleCallback: f cancelIdleCallback()
```



자바스크립트의 객체 모델

윈도우(Window) 객체 구조

BOM 브라우저 자체



DOM은 W3C와 WHATWG가 만든 **HTML 문서 조작 표준**이고, **BOM**은 브라우저 창 자체를 다루는 **객체들을 뭉뚱그려 부르는 비공식 용어**

DOM은 HTML 문서의 노드 구조와 이벤트를 정의하는 표준 객체 모델

- 브라우저객체 모델(BOM, Browser Object Model) – 공식 표준은 아님
 - JavaScript가 브라우저와 소통하기 위해서 만들어진 모델, Web Browser와 관련된 객체의 집합
 - DOM이 웹 문서를 대상으로 한다면, BOM은 브라우저 자체를 대상(브라우저 창, 주소, 기록, 화면, 실행환경)으로 함
 - 모든 개별 객체들은 최상위 객체인 window 아래 존재
 - 웹 페이지 내용을 제외한 웹 브라우저 창에 포함된 객체 요소들을 의미
 - window 속성에 속하며 document 문서가 아닌 window를 제어
- BOM 객체 종류
 - window : Browser 창 안에 존재하는 모든 요소의 최상위 객체(창 크기, 팝업)
 - location : 현재 페이지에 대한 URL 정보를 가지고 있는 객체(주소 조회 및 페이지 이동)
 - navigator : 현재 사용 중인 Web Browser 정보를 가지고 있는 객체(언어, 온라인 상태)
 - history : 사용자 방문 기록을 저장하고 있는 객체(이전, 다음 페이지 이동)
 - screen : 현재 사용 중인 화면 정보를 다루는 객체(해상도 확인)
 - document : 웹 문서에서 <body> 태그를 만나면 만들어 지는 객체이며 HTML 문서 정보 <-**DOM의 진입점**

- 문서객체 모델(DOM, Document Object Model)
 - HTML 문서를 JavaScript에서 접근하고 조작할 수 있도록 객체와 트리 구조로 표현한 모델
 - html 태그를 동적으로 제어
 - 브라우저가 html 페이지를 로드하는 과정에서 태그(Tag)들은 각기 하나의 객체로 생성
 - html 등의 문서의 내용을 노드 트리(node tree)구조의 객체로 표현
 - html과 JavaScript를 연결해주는 역할로 JavaScript를 이용해 각 요소에 접근
- 역할
 - HTML 요소 조회
 - HTML 내용 변경
 - 속성 변경
 - 요소 추가 및 삭제
 - 이벤트 등록
 - 클래스 변경

요소 조회

```
JavaScript
const title = document.querySelector("#title");
```

내용 변경

```
JavaScript
title.textContent = "JavaScript 프로그래밍";
```

요소 생성과 추가

```
JavaScript
const paragraph = document.createElement("p");

paragraph.textContent = "새롭게 생성된 문장입니다.";

document.body.appendChild(paragraph);
```

요소 삭제

```
JavaScript
paragraph.remove();
```

이벤트 등록

```
JavaScript
const button = document.querySelector("#button");
const title = document.querySelector("#title");

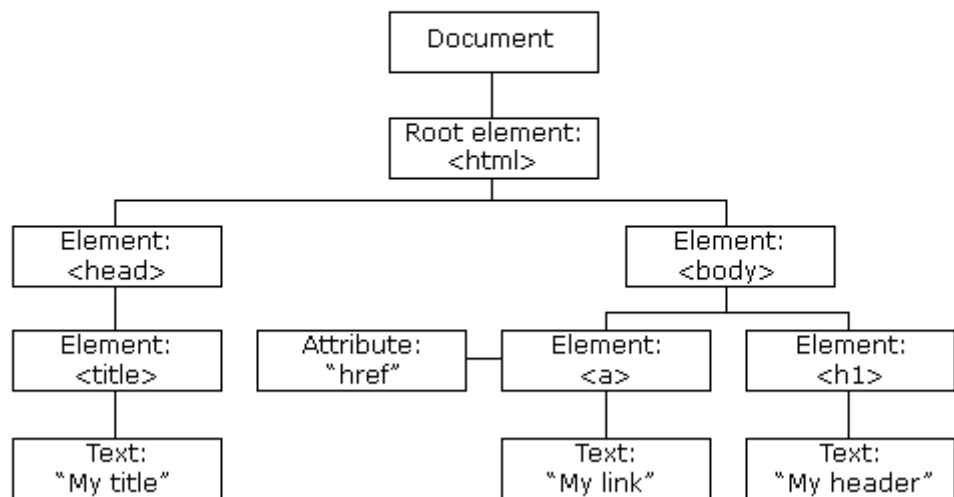
button.addEventListener("click", () => {
  title.textContent = "제목 변경 완료";
});
```

DOM 자주 사용하는 객체와 메서드

구분	예시	설명
문서 객체	<code>document</code>	현재 HTML 문서
단일 요소 조회	<code>querySelector()</code>	조건과 일치하는 첫 번째 요소
여러 요소 조회	<code>querySelectorAll()</code>	조건과 일치하는 모든 요소
요소 생성	<code>createElement()</code>	새로운 HTML 요소 생성
요소 추가	<code>appendChild()</code>	자식 요소 추가
요소 삭제	<code>remove()</code>	요소 삭제
내용 변경	<code>textContent</code>	텍스트 내용 변경
HTML 변경	<code>innerHTML</code>	내부 HTML 변경
클래스 관리	<code>classList</code>	CSS 클래스 추가·삭제
이벤트 등록	<code>addEventListener()</code>	이벤트 처리 함수 등록

- DOM 구조

The HTML DOM Tree of Objects



노드 종류	역할
Document Node	트리의 최상위에 존재하는 HTML 문서 전체
Element Node	<code><P></code> , <code><div></code> 등의 태그들
Attribute Node	<code><input></code> 등의 태그(Tag)안의 <code>name</code> , <code>value</code> 등의 속성
Text Node	HTML 문서의 모든 텍스트 표현
Comment Node	HTML 문서의 모든 주석

* HTML 속성(Attribute)은 객체 형태로 표현되지만 일반적인 자식 노드 목록에는 포함되지 않음

`<input id="name" value="Kim">`

element

attribute

- DOM 실습

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM 실습 </title>
</head>
<body>

  <h1 id="title">Hello JavaScript</h1>
  <button id="btn">변경</button>

  <script>
    const title = document.querySelector("#title");
    const button = document.querySelector("#btn");

    button.addEventListener("click", () => {
      title.textContent = "DOM이 변경되었습니다.";
    });
  </script>

</body>
</html>
```

- addEventListener 메소드**

- 구문: 요소.addEventListener(이벤트, 실행할 함수, 옵션)
- 첫번째 : 감지할 이벤트 이름을 문자열로 전달
- 두번째 : 이벤트가 발생 했을때 실행할 콜백 함수를 전달

Hello JavaScript

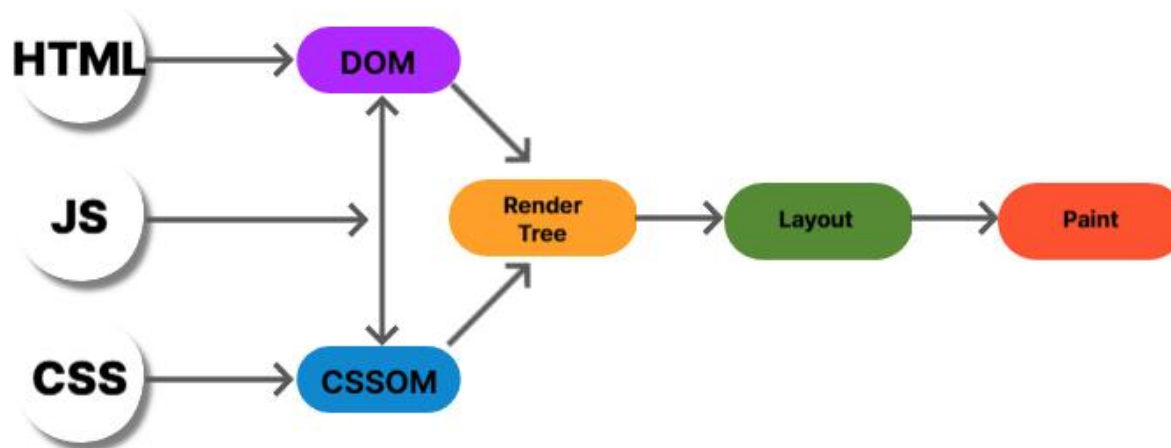
변경



DOM이 변경되었습니다.

변경

- CSS객체 모델(CSSOM, CSS Object Model)
 - DOM과 동일한 과정을 통해 객체 모델을 생성
 - DOM과 비슷해 보이지만 각 객체가 지닌 스타일 속성에 대한 값들이 트리구조로 연결
 - HTML을 파싱하여 자료를 구조화 한것을 DOM이라 하고 CSS 내용을 파싱하여 자료를 구조화한 것을 CSSOM
- 역할
 - CSS 규칙 저장 , 선택자에 맞는 요소 검색, 상속과 우선순위 계산 , 최종 적용 스타일 계산
 - JavaScript를 통한 스타일 조회와 변경, 렌더링 과정에 필요한 스타일 정보 제공
- 브라우저는 DOM과 CSSOM을 함께 사용하여 어떤 요소를 어떻게 표시할지(렌더링) 계산



- 렌더링 엔진은 DOM과 CSSOM을 실제 화면으로 변환
- HTML과 CSS는 그 자체로 화면이 아니며, 브라우저의 렌더링 과정을 거쳐 픽셀로 표현

브라우저 Rendering 과정

화면이 그려지는 순서를 와우기보다, 각 단계가 무엇을 의미하는지 이해하기



브라우저는 HTML과 CSS를 해석하고, 각 요소의 스타일과 위치를 계산한 뒤 화면을 그린다.



핵심 정리: Rendering은 HTML과 CSS를 해석한 뒤, 스타일 계산 → 레이아웃 계산 → 그리기 → 합성 과정을 거쳐 화면을 완성하는 과정이다.

ECMAScript

JavaScript 표준

JavaScript의 문법과
동작 규칙을 정의하는 표준



JavaScript Engine (V8)

JavaScript 코드를 해석하고 실행

소스 코드 → 파싱 → 실행

JavaScript 실행

예시

```
document.querySelector("h1")  
.style.color = "red";
```

1. JavaScript가 DOM API 사용
2. DOM / Style 정보 변경
3. Rendering 과정 진행
4. 화면 업데이트

Browser APIs

DOM / Event / Fetch / Timer 등
브라우저에서 제공하는 API



DOM



Event



Fetch



Timer

DOM 변경

DOM

HTML 구조를
객체화한 트리

CSSOM

CSS 스타일 규칙을
객체화한 트리

Rendering Engine (렌더링 엔진)

DOM과 CSSOM을 기반으로 화면을 그리는 과정

① HTML Parsing

HTML → DOM

HTML 구조를
객체화합니다.

② CSS Parsing

CSS → CSSOM

스타일 규칙을
객체화합니다.

③ Style Calculation

이 요소에 어떤
CSS가 적용되는가?
계산합니다.

④ Layout

어디에 위치할 것인가?
크기는 얼마인가?
계산합니다.

⑤ Paint

글자, 배경색,
태두리, 그림자 등을
그립니다.

⑥ Composite

여러 Layer를
결합하여
최종 화면을 만듭니다.

화면 (브라우저에 렌더링)

Hello, Web!

비동기 처리 흐름 (이벤트 루프)

Web API

타이머, 네트워크 요청 등을
브라우저가 처리

Task Queue (매크로태스크 큐)

setTimeout, 이벤트 콜백 등

Microtask Queue (마이크로태스크 큐)

Promise, queueMicrotask 등

Event Loop

Call Stack이 비어 있으면
Microtask → Task 순으로 실행

Call Stack

함수 호출이 쌓이는 스택
(실제 JavaScript 코드 실행)

- **DOM은 HTML 파일 자체가 아니다**
 - DOM은 브라우저가 HTML을 분석하여 **메모리에 생성한 객체 구조**입니다.
- **DOM은 화면 자체가 아니다**
 - DOM은 문서 구조이고, 실제 화면은 DOM과 CSSOM을 기반으로 Layout과 Paint 과정을 거쳐 만들어집니다.
- **CSSOM은 단순히 CSS 파일이 아니다**
 - CSSOM은 브라우저가 CSS 규칙을 분석하고 계산하기 위해 만든 **객체 모델**입니다.
- **BOM과 Web API는 완전히 같은 의미가 아니다**
 - BOM은 주로 window, location, history, navigator 등 **브라우저 관련 객체**를 지칭합니다.
 - Web API는 DOM, Fetch, Storage, Canvas, Geolocation 등 **브라우저가 제공하는 더 넓은 범위의 기능을 포함**합니다.
- **JavaScript 엔진이 DOM과 BOM을 제공하는 것은 아니다**
 - JavaScript 엔진은 JavaScript 코드를 실행하고, **DOM과 BOM은 브라우저 런타임이 제공**합니다.

브라우저에서 JavaScript가 동작하는 전체 구조 (1/2)

ECMAScript, JavaScript Engine, Browser APIs, DOM/CSSOM, Rendering Engine



JavaScript가 화면을 바꾸는 것은 단순히 코드 한 줄의 결과가 아니라, 표준 → 엔진 → 브라우저 API → 렌더링 과정이 함께 동작한 결과이다.

예제 코드와 동작 과정

JavaScript

```
document.querySelector("h1").  
style.color = "red";
```

- 1 JavaScript가 DOM API 사용
- 2 DOM / Style 정보 변경
- 3 Rendering 과정 진행
- 4 화면 업데이트



이 한 줄의 코드가 오른쪽의 전체 과정을 거쳐 화면이 바뀝니다.



ECMAScript
JavaScript 표준

문법과 동작 규칙을 정의하는
표준 명세

예시: ECMAScript 2023
(문법, 내장 객체, 동작 규칙 등)



JavaScript Engine (V8 예시)

표준을 구현하여 JavaScript
코드를 해석·실행

예시: V8 (Chrome),
SpiderMonkey (Firefox),
JavaScriptCore (Safari)



Browser APIs

DOM / Event / Fetch / Timer

브라우저가 제공하는 기능

예시: document, addEventListener,
fetch, setTimeout 등

DOM 변경



DOM

HTML 구조를 객체화한 트리



CSSOM

CSS 스타일 규칙을 객체화한 트리



Rendering Engine

DOM과 CSSOM을 기반으로 화면을 다시 그림

1 HTML Parsing
HTML → DOM

2 CSS Parsing
CSS → CSSOM

3 Style Calculation
어떤 CSS가
적용되는가?

4 Layout
이기에 위치할 것인가?
크기는 얼마인가?

5 Paint
글자·배경색·
테두리·그림자

6 Composite
여러 Layer 결합



Browser 화면

변경된 내용이 화면에 표시됨



핵심 정리

JavaScript는 엔진 위에서 실행되고, Browser API와 Rendering Engine을 통해 화면 변화로 이어진다.

브라우저에서 JavaScript가 동작하는 전체 구조 (2/2)

비동기 처리, Web API, Event Loop, Task Queue, Microtask Queue



JavaScript는 한 번에 하나씩 실행되지만, 오래 걸리는 작업은 브라우저에 맡기고 완료되면 다시 실행한다. 이때 Event Loop가 실행 순서를 조정한다.

예제 코드와 예상 결과

```
JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

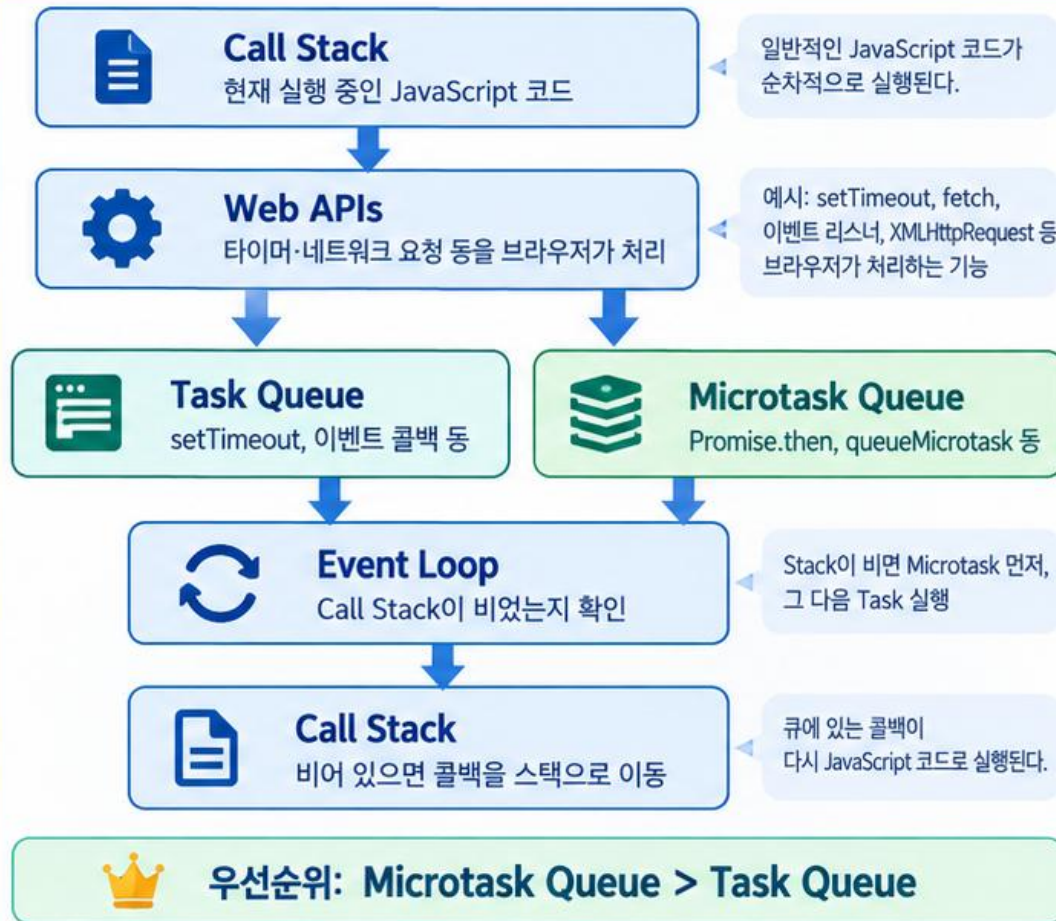
예상 출력 순서

- 1 Start
- 2 End
- 3 Promise
- 4 Timeout



동기 코드는 먼저 실행되고, Promise(Microtask)가 setTimeout(Task)보다 먼저 실행된다.

비동기 실행 흐름



핵심 설명

- 1 동기 작업 실행**
일반 코드는 Call Stack에서 순차적으로 먼저 실행된다.
- 2 비동기 작업 등록**
setTimeout은 Web API를 거쳐 Task Queue로, Promise.then은 Microtask Queue로 들어간다.
- 3 이벤트 루프 작동**
Event Loop는 Call Stack이 비어 있는지 계속 확인한다.
- 4 마이크로태스크 우선**
현재 실행이 끝난 후에는 Microtask Queue가 Task Queue보다 먼저 처리된다.



한 줄 이해

동기 코드 → Microtask → Task



핵심 정리

비동기 처리에서는 Event Loop가 Call Stack을 확인하고, Microtask Queue를 먼저 처리한 뒤 Task Queue를 실행한다.